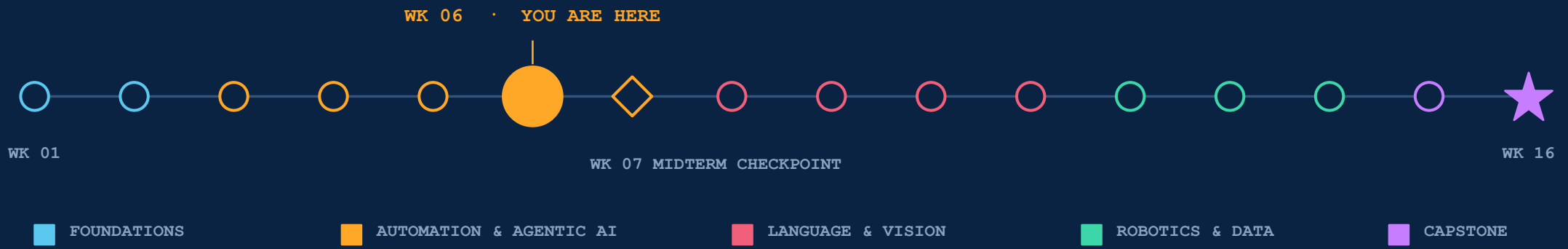
# MODEL CONTEXT PROTOCOL

WEEK 06 · AGENTIC AI II · TOOL USE & ORCHESTRATION




*Week 6 covers the open standard that lets agents use tools. Your Midterms Pt 1 project is to build one yourself.*

| | |
|---|---|
| REV | A |
| SHEETS | 1 OF 20 |
| SOURCE | WEEKLY TOPIC OUTLINE |
| SCALE | 1 SESSION |

WK.06 · OVERVIEW

# WHAT YOU WILL WALK OUT WITH

01 Explain why a model alone cannot act, and how MCP solves the N × M integration problem

02 Name the roles (host, client, server) and the three primitives (tools, resources, prompts)

03 Read a JSON-RPC exchange from initialize all the way to tools/call

04 Write a well-scoped tool with a schema an agent can actually use

05 Reason about orchestration, security and concurrency when several agents share tools

**PART A · CONCEPTS**
Slides 03-06 · The problem, what MCP is, architecture, primitives

**PART B · PROTOCOL**
Slides 07-09 · Lifecycle, transports, anatomy of a tool

**PART C · ORCHESTRATION**
Slides 10-11 · The tool-use loop and multi-agent patterns

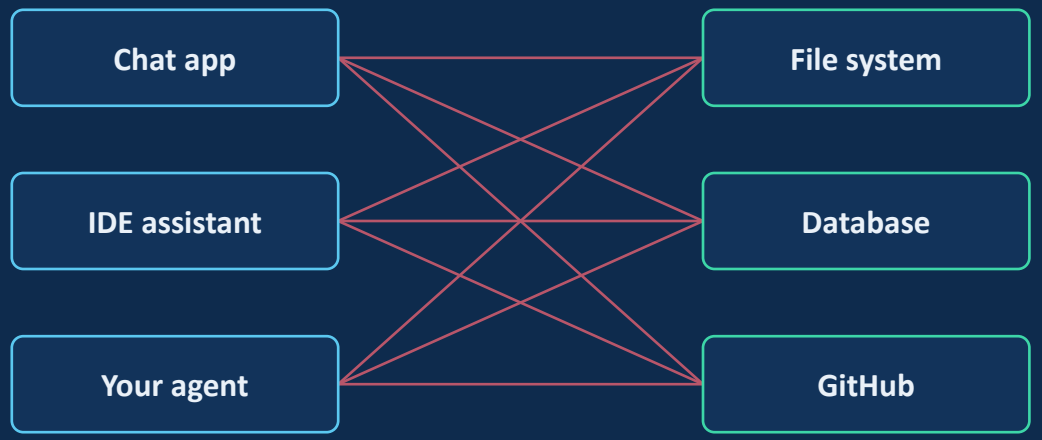
**PART D · CASE STUDY**
Slides 12-15 · My file-server connector: tools, security, protocol

**PART E · PROJECT**
Slides 16-20 · Midterms Pt 1 brief, stack, and rubric

PART A · CONCEPTS · 03

# WHY AGENTS NEED TOOLS

## WITHOUT MCP · N × M




**3 apps × 3 services = 9 custom connectors**

Every new app or service multiplies the work

## WITH MCP · N + M




**3 + 3 = 6 pieces, each written once**

Every new app or service adds a single piece

Before MCP, each vendor had its own function-calling format and every integration was hand-wired. MCP does for AI tools what USB-C did for chargers: one plug shape, any device.

PART A · CONCEPTS · 04

# WHAT IS MCP?

DEFINITION

**An open standard that lets AI applications plug into tools and data through one common interface.**

Think USB-C for AI. Build a server once and every compliant client can use it.

## NOV 2024

Introduced by Anthropic as an open specification

## JSON-RPC 2.0

Every message is a plain JSON request, response or notification

## MANY SDKS

Official libraries for TypeScript, Python and more

## ONE CONTRACT

The same server works with any compliant client

### IT IS NOT A MODEL

MCP carries requests to tools. The reasoning still happens inside the LLM.

### IT IS NOT AN AGENT FRAMEWORK

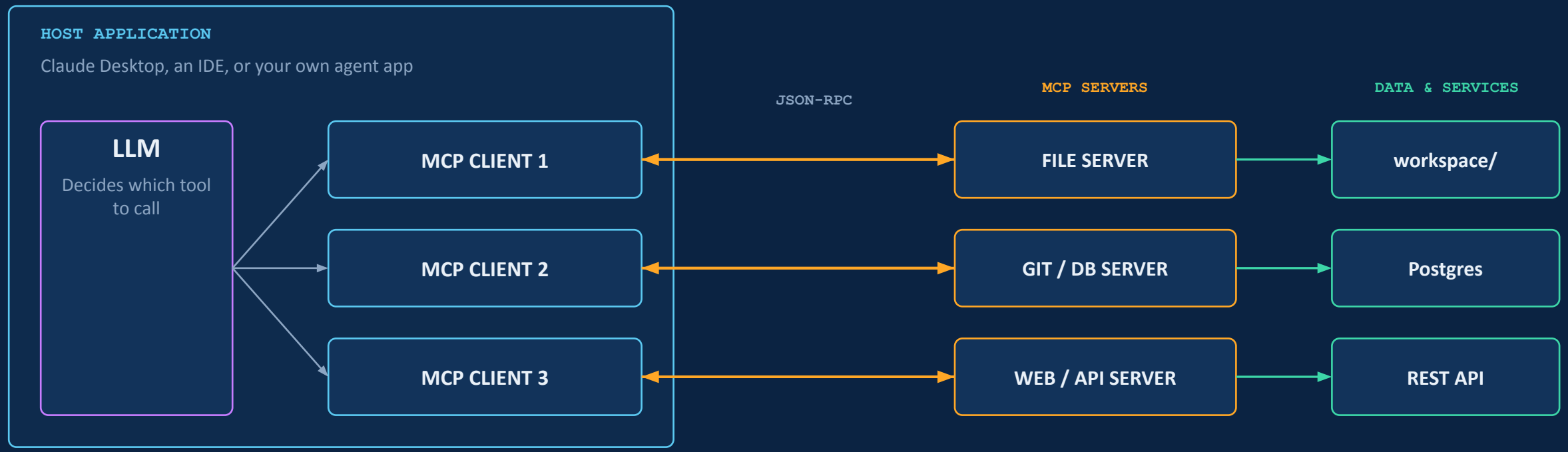
It does not plan or loop. Your agent or host does. MCP only standardizes tool access.

### IT IS A CONTRACT

Servers promise what they offer and how to call it. Clients promise to follow the schema.

PART A · CONCEPTS · 05

# ARCHITECTURE: HOST, CLIENT, SERVER




**HOST**

The app the user talks to. Owns the LLM, the conversation, and the permission prompts.

**CLIENT**

Lives inside the host. Holds exactly one connection to one server and speaks the protocol.

**SERVER**

Exposes tools, resources and prompts. Knows nothing about the LLM behind the host.

PART A · CONCEPTS · 06

# THE THREE PRIMITIVES

## TOOLS

MODEL-CONTROLLED

Actions the model can invoke. They can read, write, or change the outside world.

EXAMPLE

```
write_file(path, content)
```

USE WHEN

The agent needs to do something.

## RESOURCES

APPLICATION-CONTROLLED

Read-only context addressed by a URI. The host decides what to attach.

EXAMPLE

```
file:///projects/demo/PROJECT_STATE.md
```

USE WHEN

You want to hand over reference data without a tool call.

## PROMPTS

USER-CONTROLLED

Reusable message templates the user picks, often exposed as slash commands.

EXAMPLE

```
/review-code   file=app.js
```

USE WHEN

You want a repeatable workflow with fixed wording.

CLIENT-SIDE FEATURES · NEWER SPEC

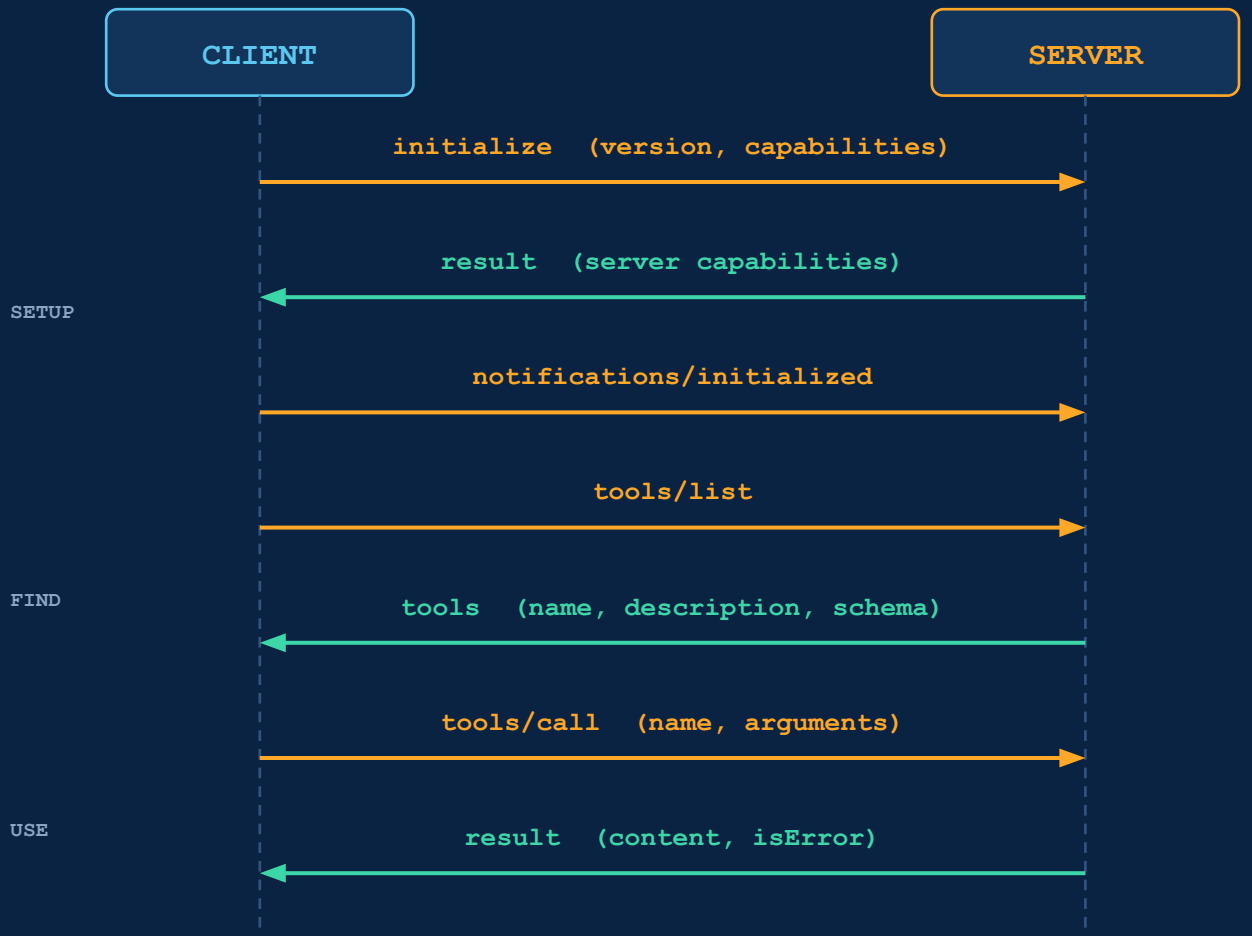
**Roots** The client tells the server which folders are in scope.

**Sampling** The server asks the host's LLM to generate text.

**Elicitation** The server asks the user for missing input.

PART B · PROTOCOL · 07

# THE PROTOCOL LIFECYCLE

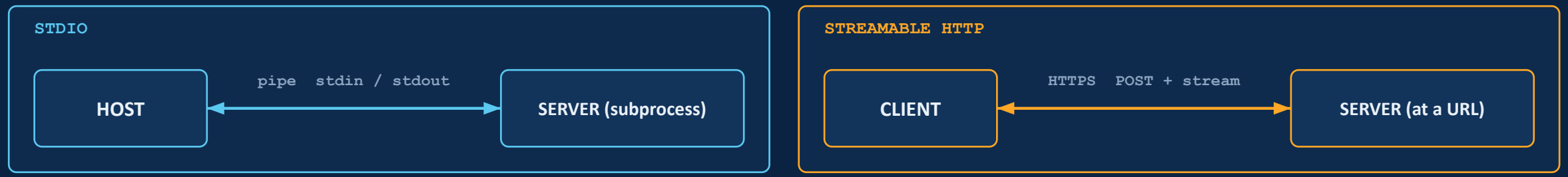



```
// client -> server
{
  "jsonrpc": "2.0",
  "id": 7,
  "method": "tools/call",
  "params": {
    "name": "read_file",
    "arguments": {
      "path": "demo/PROJECT_STATE.md"
    }
  }
}

// server -> client
{
  "jsonrpc": "2.0",
  "id": 7,
  "result": {
    "content": [
      { "type": "text", "text": "# State..." }
    ],
    "isError": false
  }
}
```

PART B · PROTOCOL · 08

# TRANSPORTS: STDIO VS HTTP




| | STDIO | STREAMABLE HTTP |
|---|---|---|
| **Where the server runs** | A subprocess the host launches on the same machine | A web service, local or remote, at a URL |
| **Who can connect** | Only the host that spawned it | Any client that can reach the URL |
| **Setup** | Add a command and its args to the host config | Deploy it, expose a URL (tunnel or cloud), give the URL to the client |
| **Auth** | Inherits your user account, nothing to configure | You must add it: tokens, a secret path, or OAuth |
| **Best for** | Personal tools, quick experiments, desktop clients | Shared servers, many devices, multi-agent work (our connector) |

*The older HTTP+SSE transport is being replaced by Streamable HTTP in the spec.*

PART B · PROTOCOL · 09

# ANATOMY OF A GOOD TOOL

```
{
  "name": "str_replace",
  "description": "Replace one exact
     occurrence of old_str in a file.
     Fails if the match is not unique.",
  "inputSchema": {
    "type": "object",
    "properties": {
      "path":    { "type": "string" },
      "old_str": { "type": "string" },
      "new_str": { "type": "string" }
    },
    "required": ["path","old_str","new_str"]
  }
}
```

**NAME** — Short verb_noun. Keep it stable, because agents and logs refer to it.

**DESCRIPTION** — The prompt the model reads. Say what it does, when to use it, and how it fails.

**INPUT SCHEMA** — JSON Schema the model fills in. Required fields, types and enums block bad calls.

**RESULT** — Content blocks plus an isError flag, so the agent can read the failure and recover.

**WEAK DESCRIPTION**

*"Edits files."*

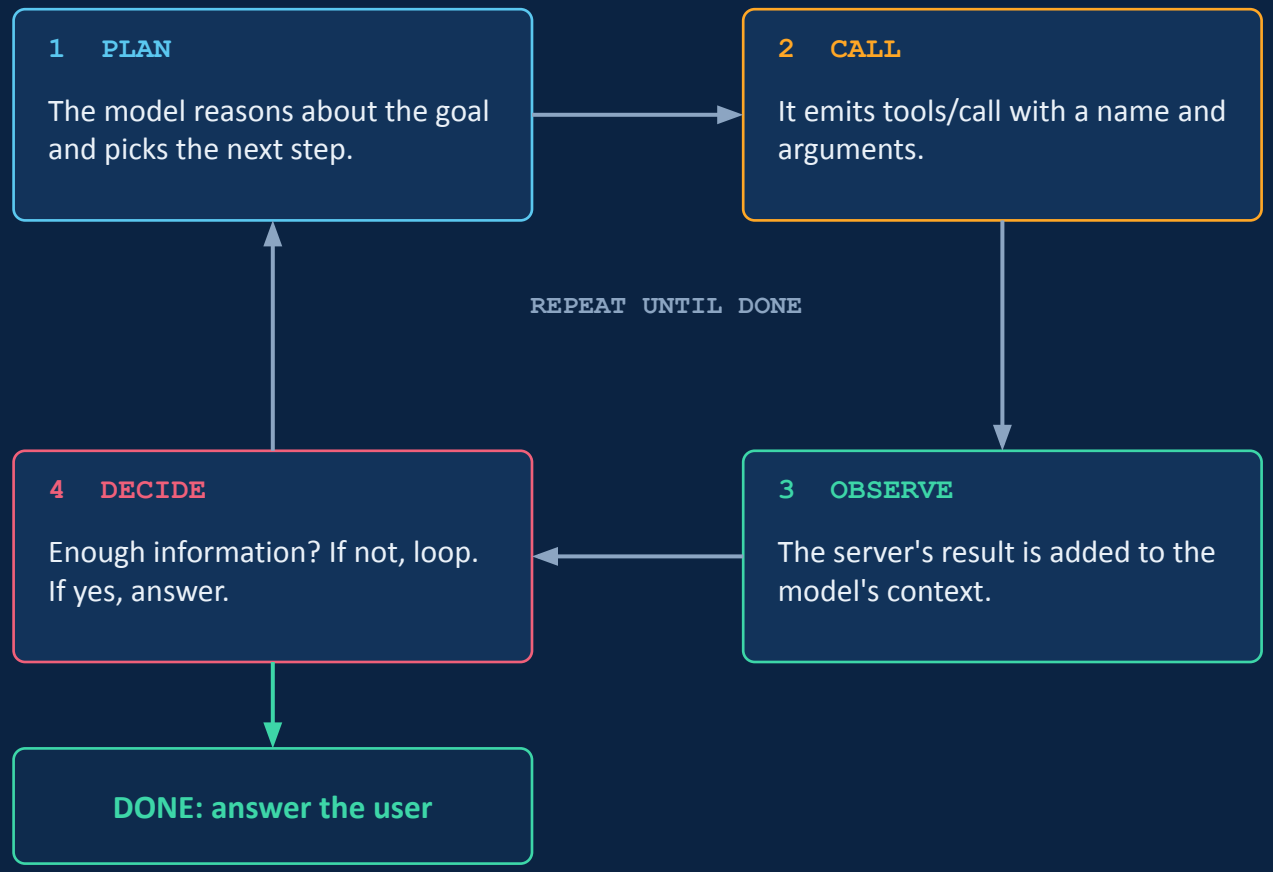
Vague. The model has to guess when to use it and what happens on failure.

**STRONG DESCRIPTION**

*"Replace one exact occurrence of old_str in a file. old_str must appear exactly once or the call fails, so widen the match with surrounding text."*

PART C · ORCHESTRATION · 10

# THE TOOL-USE LOOP




## WHERE EFFICIENCY IS WON OR LOST

**Read before you write.** Never edit blind.

**Keep results small.** Chunk, search, or map instead of dumping whole files.

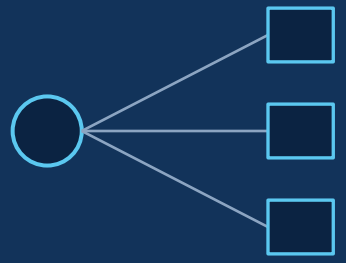
**Batch independent calls.** Do not serialize what can run together.

**Set a stop condition** and a maximum step count.

**Log every call.** You cannot improve what you cannot see.

PART C · ORCHESTRATION · 11

# ORCHESTRATION PATTERNS



## SINGLE AGENT + TOOLBOX

One agent, many tools. Start here. Most tasks need nothing more.

**RISK**

Context grows fast on long jobs.

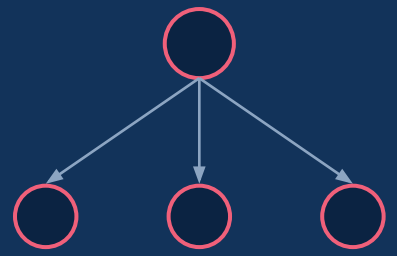


## PIPELINE

Fixed stages such as plan, code, review. Each agent sees the previous output.

**RISK**

One weak stage poisons everything after it.

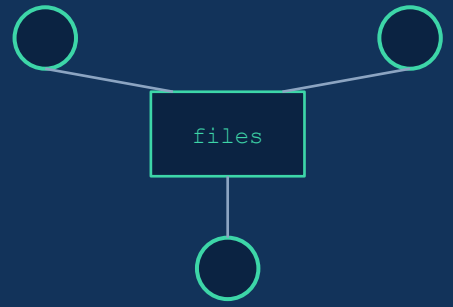


## ORCHESTRATOR + WORKERS

A lead agent splits the work and runs workers in parallel.

**RISK**

Cost multiplies and merging results is hard.




## SHARED WORKSPACE

Agents never talk directly. They leave state in files and take turns. Our connector does this.

**RISK**

Conflicting writes need locking and a protocol.

PART E · PROJECT · 16

# MIDTERMS PT 1: BUILD YOUR OWN

THE BRIEF

**Build and deploy your own MCP file-server connector, then prove it by running a real multi-step task across more than one agent session.**

This is the tool you will keep using for every project after this course: shared memory for you and your agents.

DELIVERABLES

- Working repository
- README anyone can follow
- Tool-call transcript of the demo task
- Short report and a 5-minute live demo

MINIMUM REQUIREMENTS

| | |
|---|---|
| 01 | Sandboxed workspace root. Every path is resolved against it and escapes are rejected. |
| 02 | At least 8 tools: file ops (list, read, write, str_replace, delete) plus git-lite or search. |
| 03 | Served over HTTPS through a tunnel and kept alive by a process manager. |
| 04 | Handoff protocol in place: PROJECT_STATE, PLAN_LOG, CHECKPOINTS, DECISIONS. |
| 05 | Connected to a real MCP client and tested on one multi-step task across 2 or more sessions. |
| 06 | A short report on your stack, tool design choices, and security tradeoffs. |

**STRETCH** if_version locking, search_code, a repo map, token auth.

PART E · PROJECT · 17

# STACK AND MILESTONES

| LAYER | REFERENCE CHOICE | ALTERNATIVES |
|---|---|---|
| **Runtime and SDK** | Node.js with the official TypeScript MCP SDK | Python with the official SDK or FastMCP |
| **Transport** | Streamable HTTP | stdio for local-only testing |
| **Validation** | zod schemas on every tool input | pydantic in Python |
| **Exposure** | ngrok with a static domain | Cloudflare Tunnel, or a small VPS |
| **Process manager** | pm2 | systemd or Docker |
| **Version control** | git on the host | Any git CLI setup |
| **Testing** | MCP Inspector, then a real client | Scripted JSON-RPC calls |

*Pick one stack and justify it in your report. A better-reasoned alternative scores the same as the reference.*

**M1 SKELETON**

Server boots and one tool, list_files, answers in MCP Inspector.

**M2 SANDBOX**

Core file tools plus path-escape protection and tests.

**M3 DEPLOY**

Tunnel and process manager running. A real client connects.

**M4 PROTOCOL**

Handoff files, git-lite tools, and guards against bad writes.

**M5 PROVE IT**

Multi-agent run, report, and demo at the Week 7 checkpoint.

PART E · PROJECT · 18

# RUBRIC: HOW YOU WILL BE GRADED

POINTS BY CRITERION · 100 TOTAL

| | |
|---|---|
| A Agent efficiency | 20 |
| B Agent and tool management | 20 |
| C Tool design and implementation | 20 |
| D Tech stack and architecture | 15 |
| E Security and sandboxing | 10 |
| F Documentation, practice, demo | 15 |

*Amber rows: 40 points on how well agents and their tools are run.*

## GRADE BANDS

| | |
|---|---|
| 90–100 | Exemplary |
| 80–89 | Proficient |
| 70–79 | Developing |
| Below 70 | Revise and resubmit |

## EVIDENCE TO SUBMIT

- Repository link and README
- Transcript of one multi-step task
- PLAN_LOG and CHECKPOINT files
- 5-minute live demo

**Efficiency is judged from your transcript and logs, not from claims.**

PART E · PROJECT · 19

# RUBRIC I: AGENTS AND TOOLS

| CRITERION | EXEMPLARY | PROFICIENT | DEVELOPING | BEGINNING |
|---|---|---|---|---|
| **A Agent efficiency**<br>20 pts<br>Evidence: tool-call transcript, PLAN_LOG | 18-20<br>Multi-step task finishes in few, purposeful calls. Reads before writing, batches independent calls, keeps outputs small with chunking or search, and never retries without new information. | 15-17<br>Task completes. A few redundant or oversized calls, and errors are handled with only minor waste. | 11-14<br>Task completes only after repeated or looping calls. Often reads whole files. Weak stop condition. | 0-10<br>Task not completed, or runaway loops. No sign of planning before acting. |
| **B Agent and tool management**<br>20 pts<br>Evidence: handoff files, logs, config | 18-20<br>Each agent has a clear role and only the tools it needs. Destructive tools are gated. Handoff through PROJECT_STATE, PLAN_LOG and CHECKPOINTS carries context across 2+ sessions. Calls are logged. | 15-17<br>Roles and tool access mostly scoped. Handoff files are used but sometimes stale or incomplete. | 11-14<br>Every agent gets every tool. Handoff is informal and logs are partial. | 0-10<br>No role separation or handoff. Agents overwrite each other's work. |
| **C Tool design and implementation**<br>20 pts<br>Evidence: source code, Inspector run | 18-20<br>8+ tools, each single-purpose with a clear description and a strict schema. Helpful error messages through isError. Verified in MCP Inspector and with a real client. | 15-17<br>8+ tools work. Descriptions or schemas are occasionally vague. Errors mostly handled. | 11-14<br>5-7 tools, or some fail edge cases. Weak schemas. Errors crash the server or return raw stack traces. | 0-10<br>Fewer than 5 working tools, or tools were never verified. |

PART E · PROJECT · 20

# RUBRIC II: BUILD AND CRAFT

| CRITERION | EXEMPLARY | PROFICIENT | DEVELOPING | BEGINNING |
|---|---|---|---|---|
| **D Tech stack and architecture**<br>15 pts<br>Evidence: report, repo layout, config | 14-15<br>Stack choices (runtime, SDK, transport, tunnel, process manager) are justified in writing. Clean separation of server, tools and config. Secrets in environment variables. Survives a restart. | 11-13<br>Sensible stack with partial justification. Minor hardcoded config. | 8-10<br>Works on one machine only. No process manager. Hardcoded secrets or paths. | 0-7<br>Cannot be rebuilt from the README. |
| **E Security and sandboxing**<br>10 pts<br>Evidence: path tests, endpoint check | 9-10<br>Path traversal tests pass and the sandbox root is enforced. The public endpoint needs a secret or token. Destructive actions are gated or logged. Threats are noted in the report. | 7-8<br>Sandbox enforced and auth present, with limited testing. | 5-6<br>Sandbox can be bypassed, or the endpoint is open. | 0-4<br>Agents can reach files outside workspace/, or there is no protection. |
| **F Documentation, practice, demo**<br>15 pts<br>Evidence: README, git log, live demo | 14-15<br>A classmate can reproduce the setup from the README. Git history is meaningful. Live demo of a multi-step task goes smoothly and design tradeoffs are explained well in Q&A. | 11-13<br>README is complete. Demo works with small gaps. Q&A mostly solid. | 8-10<br>Docs are thin. Demo is scripted and fragile. Limited Q&A. | 0-7<br>No usable docs or demo. |